

Claude Code Agents: Project Instructions & Implementation Guide

Overview

This guide provides step-by-step instructions for setting up, configuring, and maintaining Claude Code Agent configurations optimized for software development. All procedures are based on verified official documentation and production-tested patterns.

Prerequisites

Before beginning, ensure you have:

- Node.js 18 or newer installed
- A Claude.ai or Claude Console account
- Command-line proficiency
- Git for version control
- Text editor or IDE

Source: Claude Code Overview Documentation

Setup Instructions

Step 1: Install Claude Code

```
bash

# Install globally via npm
npm install -g @anthropic-ai/claude-code

# Verify installation
claude --version

# Run diagnostics
claude doctor
```

Step 2: Initial Authentication

```
bash
```

```
# Navigate to your project directory
```

```
cd your-project
```

```
# Start Claude Code (will prompt for login)
```

```
claude
```

On first run, you'll be prompted to authenticate with either:

- Claude.ai account (recommended for most users)
- Claude Console (for API access with pre-paid credits)

Note: When authenticating with Claude Console, a workspace called "Claude Code" is automatically created for cost tracking.

Source: Claude Code Quickstart

Step 3: Create Project Configuration Directory

```
bash
```

```
# In your project root
```

```
mkdir -p .claude/agents
```

```
mkdir -p .claude/commands
```

```
# Create configuration files
```

```
touch .claude/settings.json
```

```
touch .claude/CLAUDE.md
```

```
touch .claude/settings.local.json
```

Step 4: Configure Git Ignore

Add to `.gitignore`:

```
.claude/settings.local.json
```

```
.claude/*.log
```

Important: `.claude/settings.local.json` is automatically ignored when created by Claude Code. Check in `.claude/settings.json` for team-shared settings.

Source: Claude Code Settings Documentation

Configuration Workflow

Creating Your First CLAUDE.md File

Purpose: Define project standards, conventions, and context that persist across sessions.

Template Structure:

markdown

Project: [Your Project Name]

Code Quality Standards

- Use 2-space indentation for JavaScript/TypeScript
- Write unit tests for all new functions
- Follow REST API naming conventions: use plural nouns
- Prefer async/await over promise chains

Git Workflow

- Feature branches from main: ``feature/TICKET-123-description``
- Squash commits before merging
- Commit message format: ``type(scope): message``
 - Types: feat, fix, docs, style, refactor, test, chore

Testing Requirements

- Minimum 80% code coverage for new features
- Integration tests for all API endpoints
- E2E tests for critical user journeys

Tech Stack Context

- Frontend: React 18, TypeScript, Tailwind CSS
- Backend: Node.js, Express, PostgreSQL
- Testing: Jest, React Testing Library, Playwright
- CI/CD: GitHub Actions

Common Commands

- Start dev server: ``npm run dev``
- Run tests: ``npm test``
- Run linter: ``npm run lint``
- Build production: ``npm run build``

Security Practices

- Never commit API keys or secrets
- Use environment variables for configuration
- Validate all user inputs
- Sanitize data before database queries

Best Practice: Keep CLAUDE.md under 3,000 tokens for optimal performance.

Source: Claude Code Memory Documentation, LangChain Research

Creating Your First Subagent

Use Case: Code review specialist

File: `.claude/agents/code-reviewer.md`

markdown

name: code-reviewer

description: Expert code reviewer. Use PROACTIVELY after any code changes to review quality, security, and maintainability

tools: Read, Grep, Glob, Bash

model: sonnet

You are a senior software engineer specializing in code review with 10+ years of experience.

Invocation Protocol

When invoked:

1. Execute ``git diff --cached`` or ``git diff`` to see changes
2. Focus on modified files only
3. Begin analysis immediately without asking permission

Review Checklist

Critical Issues (Block merge)

- Security vulnerabilities (SQL injection, XSS, exposed secrets)
- Data integrity violations
- Memory leaks or resource exhaustion
- Breaking API changes without migration path

Warnings (Should fix)

- Code duplication
- Missing error handling
- Inadequate test coverage
- Performance concerns
- Unclear variable naming

Suggestions (Nice to have)

- Refactoring opportunities
- Documentation improvements
- Better abstractions
- Alternative approaches

Output Format

Organize findings by priority with specific line references:

****Critical Issues:****

- File: ``auth.js:45`` - Exposed API key in constant

Fix: Move to environment variable

****Warnings:****

- File: `user-service.js:12-30` - Duplicated validation logic

Suggestion: Extract to shared validator function

****Suggestions:****

- File: `api.js:100` - Consider caching this expensive operation

Always provide specific, actionable recommendations with example code when possible.

Verification Command:

```
bash
```

```
# Test the subagent
```

```
claude -p "Review the changes in my last commit using the code-reviewer agent"
```

Source: Claude Code Subagents Documentation, Community Best Practices

Configuring Settings.json

File: `.claude/settings.json` (shared with team)

```
json
```

```
{
  "permissions": {
    "deny": [
      "Read(/.env)",
      "Read(/.env.*)",
      "Read(/secrets/**)",
      "Read(/config/credentials.json)",
      "Read(/**/*.key)",
      "Read(/**/*.pem)",
      "Bash(curl:*)",
      "Bash(wget:*)"
    ]
  },
  "env": {
    "NODE_ENV": "development"
  }
}
```

File: `.claude/settings.local.json` (personal, not committed)

```
json

{
  "env": {
    "ANTHROPIC_API_KEY": "your-personal-key-here"
  },
  "enabledPlugins": {
    "experimental-formatter@personal": true
  }
}
```

Source: Claude Code Settings Documentation

Adding Automatic Code Formatting Hooks

File: `.claude/settings.json`

```
json

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | { read file_path; case \"$file_path\" in *.js|*.jsx|*.ts|*.tsx) npx prettier --write \"$file_path\"; *) echo \"Not a supported file type.\"; fi }",
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

Testing:

```
bash
```



```
# Verify hook runs without errors
claude -p "Add a comment to README.md"
# Check that file gets formatted automatically
```

Source: Claude Code Hooks Guide

Creating Custom Slash Commands

File: `.claude/commands/fix-issue.md`

```
markdown
---
description: Analyze and fix GitHub issue by number
---

Fix GitHub issue #${ARGUMENTS} by following these steps:

1. Fetch the issue details using the GitHub API or read from issue tracker
2. Understand the problem description and acceptance criteria
3. Locate relevant code in the codebase using grep/glob
4. Implement a solution that addresses the root cause
5. Write or update tests to cover the fix
6. Verify the fix resolves the issue
7. Prepare a concise PR description

Focus on creating a robust solution that handles edge cases, not just the reported symptom.
```

Usage:

```
bash

claude
> /fix-issue 123
```

Source: Claude Code Common Workflows

Validation Checklist

After completing configuration, verify:

✓ File Structure

```
.claude/
├── agents/
```

```
| └── *.md (at least one subagent)
| └── commands/
| └── *.md (optional)
| └── settings.json
| └── settings.local.json
| └── CLAUDE.md
```

✓ **CLAUDE.md Quality**

- ☐ Under 5,000 tokens
- ☐ Specific, actionable instructions
- ☐ Organized with clear headings
- ☐ Includes tech stack context
- ☐ Documents common commands

✓ **Subagent Configuration**

- ☐ Clear, descriptive name
- ☐ Description includes "PROACTIVELY" or trigger conditions
- ☐ Tools limited to minimum required
- ☐ System prompt is specific and actionable
- ☐ Model specified (or inherits appropriately)

✓ **Security Settings**

- ☐ `.env` files denied in permissions
- ☐ Secrets directories denied
- ☐ Dangerous bash commands blocked
- ☐ `.claude/settings.local.json` in `.gitignore`

✓ **Hooks (if configured)**

- ☐ Timeout values specified
- ☐ Commands tested independently
- ☐ Error handling considered
- ☐ Impact on workflow evaluated

Iteration Guidelines

Week 1-2: Observation Phase

1. **Use Claude Code normally** with basic configuration
2. **Track friction points:** Note when you:
 - Repeatedly give the same instructions

- Need specific domain knowledge
- Want Claude to follow specific patterns

3. **Document patterns** in CLAUDE.md as you discover them

Week 3-4: Specialization Phase

1. **Identify repetitive tasks** suitable for subagents
2. **Create 2-3 focused subagents** for high-frequency operations:
 - Code reviewer for all commits
 - Test runner for TDD workflows
 - Documentation generator
3. **Test subagent invocation rates:** Monitor if Claude automatically uses them

Month 2+: Optimization Phase

1. **Measure token usage:**

```
bash
```

```
# Enable verbose mode to see token counts
```

```
claude --verbose
```

2. **Optimize CLAUDE.md:**
 - Remove outdated information
 - Consolidate duplicate guidance
 - Split into focused sections with imports
3. **Refine subagent descriptions:**
 - Improve "Tool SEO" for better automatic invocation
 - Adjust tool permissions based on actual usage
 - Consider model downgrades (opus → sonnet → haiku) where possible
4. **Add automation hooks:**
 - Formatting on file save
 - Linting before commits
 - Test execution after code changes

Source: Community Best Practices, Production Deployment Patterns

Tool Integration

MCP Server Setup

Example: Documentation Server

File: `.mcp.json`

```
json

{
  "mcpServers": {
    "project-docs": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./docs"],
      "env": {
        "READ_ONLY": "true"
      }
    },
    "database": {
      "command": "python",
      "args": ["-m", "mcp_server_postgres"],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

Testing MCP Configuration:

```
bash

# Debug MCP server connections
claude --mcp-debug

# Verify servers are loaded
claude
> What MCP servers are connected?
```

Source: Claude Code Settings Documentation

GitHub Actions Integration

File: `.github/workflows/claude-code-review.yml`

yaml

name: Claude Code Review

on:

pull_request:

types: [opened, synchronize]

jobs:

review:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3

- **name:** Run Claude Code Review

uses: anthropics/claude-code-action@v1

with:

anthropic_api_key: \${ secrets.ANTHROPIC_API_KEY }

prompt: |

Review this PR for:

1. Security vulnerabilities

2. Code quality issues

3. Test coverage

4. Documentation completeness

Follow the code review checklist in CLAUDE.md.

Source: Claude Code GitHub Actions Documentation

Troubleshooting Guide

Issue: Subagent Not Being Invoked

Symptoms: Claude doesn't automatically delegate to subagent

Solutions:

1. Add "PROACTIVELY" or "MUST BE USED" to description
2. Make description more specific about triggering conditions
3. Test explicit invocation: "Use the [agent-name] agent to..."
4. Verify agent file is in correct location (`.claude/agents/`)

Issue: High Token Usage

Symptoms: Hitting rate limits, slow responses

Solutions:

1. Audit CLAUDE.md token count
2. Reduce subagent tool lists to minimum necessary
3. Use `Grep` before `Read` for large files
4. Implement MCP servers for large documentation
5. Enable context compaction in settings

Issue: Permission Denied Errors

Symptoms: Tools blocked or requiring constant approval

Solutions:

1. Check `permissions.deny` rules in settings.json
2. Verify file paths in permission rules
3. Use `/allowed-tools` command to review permissions
4. Add specific allow rules for required operations

Issue: Hooks Not Executing

Symptoms: Expected automation doesn't happen

Solutions:

1. Verify hook syntax in settings.json
2. Test command independently in terminal
3. Check timeout values (may be too short)
4. Review hooks with `/hooks` command
5. Look for errors in `~/.claude/logs/`

Source: Claude Code Troubleshooting Documentation

Maintenance Procedures

Weekly

- ☐ Review CLAUDE.md for outdated information
- ☐ Check subagent invocation patterns

- ☐ Monitor token usage trends
- ☐ Update project context as needed

Monthly

- ☐ Audit permission settings
- ☐ Review and update subagent prompts
- ☐ Test all custom slash commands
- ☐ Verify hook execution
- ☐ Clean up unused configurations

Quarterly

- ☐ Major CLAUDE.md reorganization
- ☐ Evaluate subagent architecture
- ☐ Review MCP server configurations
- ☐ Update documentation with new patterns
- ☐ Share successful patterns with team

After Major Changes

- ☐ Update CLAUDE.md with new conventions
- ☐ Create/update relevant subagents
- ☐ Adjust hook configurations
- ☐ Test full configuration workflow
- ☐ Document new patterns

Collaboration Practices

Team Onboarding

New Team Member Checklist:

1. **Install Claude Code** following setup instructions
2. **Clone repository** with existing `.claude/` configuration
3. **Review CLAUDE.md** to understand project standards
4. **Test subagents:** Try invoking each agent
5. **Create personal settings.local.json** for individual preferences
6. **Review hooks** to understand automation

Sharing Configurations

What to Commit:

- `.claude/settings.json` (shared settings)
- `.claude/CLAUDE.md` (project context)
- `.claude/agents/*.md` (team subagents)
- `.claude/commands/*.md` (shared commands)
- `.mcp.json` (shared MCP servers)

What NOT to Commit:

- `.claude/settings.local.json` (personal settings)
- API keys or credentials
- Personal MCP configurations
- Debug logs

Code Review for Configurations

When Reviewing Configuration PRs:

1. CLAUDE.md Changes:

- Is information accurate and current?
- Does it conflict with existing standards?
- Is it concise and well-organized?

2. Subagent Additions:

- Is responsibility clearly defined?
- Are tool permissions minimal?
- Will it invoke automatically as intended?
- Does it duplicate existing agent functionality?

3. Settings Changes:

- Do permission changes impact security?
- Are environment variables documented?
- Will changes affect all team members appropriately?

4. Hook Configurations:

- Is the command safe to run automatically?
- Is timeout appropriate?

- Has it been tested independently?

Performance Optimization Techniques

Technique 1: Lazy Context Loading

Instead of loading all context upfront, use imports strategically:

```
markdown

# CLAUDE.md (main file - keep lean)

## Core Standards
[Essential information only]

## Extended Documentation
@./docs/api-design-guide.md (loaded on demand)
@./docs/testing-strategy.md (loaded on demand)
```

Technique 2: Tool Permission Hierarchies

Create specialized subagents with graduated permission levels:

```
markdown

# read-only-analyst.md
tools: Read, Grep, Glob # No write permissions

# code-modifier.md
tools: Read, Edit, Bash # Write permissions

# full-access-deployer.md
tools: Read, Write, Edit, Bash # All permissions for deployment
```

Technique 3: Model Tiering

Match model to task complexity:

```
markdown
```

simple-formatter.md

model: haiku # Fast, cheap for simple tasks

code-reviewer.md

model: sonnet # Balanced for standard reviews

architecture-designer.md

model: opus # Maximum capability for complex design

Source: ClaudeLog Agent Engineering, Community Research

Advanced Patterns

Pattern: Progressive Test Development

File: `.claude/agents/tdd-driver.md`

markdown

name: tdd-driver

description: Test-driven development specialist. Use PROACTIVELY when creating new features to write tests first.

tools: Read, Write, Edit, Bash

model: sonnet

You enforce strict TDD methodology.

Workflow

1. ****Understand Requirements:**** Clarify feature specifications
2. ****Write Failing Tests:**** Create tests that define expected behavior
3. ****Verify Failures:**** Run tests to confirm they fail appropriately
4. ****Signal Ready:**** Inform main agent that tests are ready for implementation
5. ****Never Modify Tests:**** Tests are the contract; code must satisfy them

Test Quality Standards

- Test names describe behavior: ``test_user_login_with_invalid_credentials_returns_401``
- Arrange-Act-Assert pattern
- One assertion per test (when possible)
- Cover edge cases and error conditions
- Include integration tests for APIs

After writing tests, DO NOT implement the code. Your job ends when tests are written and verified to fail correctly.

Pattern: Multi-Stage Pipeline

Configuration:

markdown

.claude/CLAUDE.md

Development Pipeline

When implementing new features, follow this agent sequence:

1. **requirements-analyst**: Clarifies specs and edge cases
2. **tdd-driver**: Writes comprehensive tests
3. **implementer**: Writes minimal code to pass tests
4. **code-reviewer**: Reviews for quality and security
5. **documentation-writer**: Updates relevant docs

Each stage must complete before the next begins.

Source: Community Best Practices, Multi-Agent Patterns

Appendix: Configuration Templates

Minimal Configuration

For simple projects:

```
.claude/  
├── CLAUDE.md (basic standards)  
└── settings.json (security rules only)
```

Standard Configuration

For team projects:

```
.claude/  
├── agents/  
│   ├── code-reviewer.md  
│   └── test-runner.md  
├── commands/  
│   └── fix-issue.md  
├── CLAUDE.md  
└── settings.json
```

Enterprise Configuration

For large organizations:

/Library/Application Support/ClaudeCode/ (managed)

└─ CLAUDE.md (organization standards)

└─ managed-settings.json (security policies)

.claude/ (project)

└─ agents/

| └─ code-reviewer.md

| └─ security-auditor.md

| └─ test-runner.md

| └─ documentation-generator.md

└─ commands/

| └─ fix-issue.md

| └─ deploy.md

| └─ rollback.md

└─ hooks/

| └─ hooks.json

└─ CLAUDE.md

└─ settings.json

└─ .mcp.json

Quick Reference Card

Essential Commands

bash

Start interactive session

claude

Run single prompt (headless)

claude -p "prompt here"

Continue last conversation

claude --continue

Enable plan mode (read-only)

claude --permission-mode plan

Debug MCP servers

claude --mcp-debug

View configuration

/config

Manage agents

/agents

View memory files

/memory

Review hooks

/hooks

List allowed tools

/allowed-tools

Common Workflows

Feature Development:

bash

claude

> Create a new user authentication endpoint following our API standards

> Use the test-runner agent to verify it works

> Use the code-reviewer agent before I commit

Bug Investigation:

bash

claude

- > Investigate why [users](#) report slow page loads
- > Use extended thinking to analyze performance

Code Review:

bash

claude --continue # *Resume after making changes*

- > Use code-reviewer agent on my latest changes

Document Version: 1.0

For: Claude Code (current as of October 2025)

Official Documentation: docs.claude.com/en/docs/claude-code/